ADAMS Normalization Before-and-After Backup Guide

UNF to 1NF to 2NF to 3NF | What changed, why it changed, and how to explain it during defense

Main point: For normalization, a professor usually expects the database/table transformation, not the system UI. The UI screenshots can support the presentation, but the real evidence is the movement from one wide, repetitive table to several related database tables.

1. What the Professor May Be Looking For

A normalization discussion should clearly answer three questions:

- Before: What was wrong with the original table or logbook-style record?
- After: What tables were created or separated at each normalization stage?
- Reason: Why did the change improve the database design and the ADAMS system?

For ADAMS, the concern is not only appearance. The concern is that student information, category data, requirements, statuses, file uploads, users, logs, print history, password reset records, and archive history should not be mixed into one table. A cleaner relational design is needed so the system can search, monitor, verify, print, and archive records accurately.

2. Quick Visual Flow: What Was Updated

Stage	Database View	Ano ang Nabago / What Changed	Concern Fixed
UNF	One wide monitoring table	Student details + category + multiple Requirement/Status/File columns in one row	Repeating groups, redundant data, difficult tracking
1NF	One requirement per row	Requirement1/Requirement2 columns become multiple rows	Atomic values; no repeating requirement columns
2NF	Core tables separated	students, student_categories, requirement_types, category_requirements, student_requirements	Less duplication; correct checklist per category
3NF	Operational/support tables separated	users, documents, activity_log, print_logs, password_reset_tokens, archived_students	Audit trail, security, reporting, archiving, maintainability

3. UNF - Original Unnormalized View

In UNF, the record looks like a manual spreadsheet or logbook. It stores multiple requirements inside one student row. This is the “before” view.

Student Name	Category	Requirement 1	Status 1	File 1	Requirement 2	Status 2	File 2	Remarks
Reyes, Ana	Freshman	Birth Certificate	SUBMITTED	birth.pdf	Valid ID	PENDING	N/A	For verification
Santos, Mark	Transferee	Transcript	PENDING	N/A	Good Moral	SUBMITTED	goodmoral.pdf	Incomplete

Stage	Concern Before Update	Ano ang Nabago / What Changed	Why It Needed to Change / Result
UNF	Requirements are stored as repeated columns: Requirement 1, Status 1, File 1, Requirement 2, Status 2, File 2. If a category has more requirements, more columns must be added.	Identified that the record is too wide and mixes student profile, category, requirement list, requirement status, and upload file data in one row.	This needed to change because the design creates duplicate data, hard-to-maintain columns, and difficulty checking missing requirements.

Defense line: “The UNF problem is that the database acts like a spreadsheet. It is not flexible because requirements repeat as columns.”

4. 1NF - Repeating Groups Removed

In 1NF, each field should contain one value only, and repeating groups must be removed. The change is that each requirement is now placed in a separate row.

Student Name	Category	Requirement	Status	File Name	Uploaded By	Upload Date
Reyes, Ana	Freshman	Birth Certificate	SUBMITTED	birth.pdf	admin	2026-06-20
Reyes, Ana	Freshman	Valid ID	PENDING	N/A	admin	
Santos, Mark	Transferee	Transcript	PENDING	N/A	registrar	
Santos, Mark	Transferee	Good Moral	SUBMITTED	goodmoral.pdf	registrar	2026-06-20

Stage	Concern Before Update	Ano ang Nabago / What Changed	Why It Needed to Change / Result
1NF	UNF contains repeating requirement columns and non-atomic groups.	Converted Requirement 1/2, Status 1/2, and File 1/2 into separate rows. Each row now represents one student requirement record.	This fixes repeating groups and makes the requirement/status/file values atomic. However, student name and category are still repeated across rows, so further normalization is needed.

Defense line: “In 1NF, we did not yet create all final tables. We first corrected the repeated columns by making one requirement per row.”

5. 2NF - Core Data Separated by Purpose

In 2NF, the repeated rows from 1NF are separated into tables based on their real purpose. Student profile data should be stored once. Category names should be stored once. Requirement names should be stored once. The student requirement status should be stored separately because it changes per student and per requirement.

2NF Table	Primary Key	What Was Moved Here	Why It Needed to Be Separated
students	student_id	Student profile fields: name, email, phone, birth_date, category_id	Student identity should be stored once and not repeated for every requirement.
student_categories	category_id	category_name, sort_order	Category labels like Freshman or Transferee should be reusable.
requirement_types	requirement_id	requirement_name, description	Requirement names like Birth Certificate or Valid ID should be stored once.
category_requirements	category_id + requirement_id	Defines which requirements apply to each category	This bridge table solves the problem of different checklists per category.
student_requirements	student_requirement_id	student_id, requirement_id, status, file_name, file_path, uploaded_by, upload_date	This records the actual status and upload details per student requirement.

Stage	Concern Before Update	Ano ang Nabago / What Changed	Why It Needed to Change / Result
2NF	After 1NF, student and category details are still repeated in many rows. Requirement names are also repeated for every student.	Separated student data, category data, requirement master list, category checklist mapping, and actual student requirement records.	This reduces redundancy and allows ADAMS to apply the correct checklist per student category without rewriting the same requirements repeatedly.

Defense line: “In 2NF, we separated what belongs to the student, what belongs to the category, what belongs to the requirement master list, and what belongs to the student’s actual submitted requirement.”

6. 3NF - Final Operational Database Structure

In 3NF, data that does not directly belong inside the student profile or student requirement record is moved into separate support tables. This prevents transitive dependency and keeps operational information clean.

3NF Final Table	Key Fields	Ano ang Nabago / What Changed	Why It Needed to Be Separated
users	user_id, username, role	Stores Superadmin and Staff accounts separately	User account data should not be repeated inside each student or upload record.
activity_log	log_id, user_id	Stores actions such as ADD, UPDATE, DELETE, UPLOAD, ARCHIVE	Accountability is a separate concern from student profile data.
print_logs	log_id, ref_no	Stores printed reports and reference numbers	Printing history should be recorded without changing student profile fields.
password_reset_tokens	id, user_id, token	Stores reset tokens, expiry, and usage status	Security and password recovery should be separated from user profile fields.
documents	document_id, student_pk	Stores document records linked to a student	Document metadata can be multiple per student and should not be mixed with the student table.
archived_students	archive_id, student_id	Stores historical snapshots of removed/inactive student records	Archiving preserves history without deleting the original meaning of records.

Stage	Concern Before Update	Ano ang Nabago / What Changed	Why It Needed to Change / Result
3NF	Operational data such as user actions, print history, reset tokens, documents, and archive history should not be stored inside the main student table.	Moved these supporting concerns into their own tables: users, activity_log, print_logs, password_reset_tokens, documents, and archived_students.	This improves accountability, security, reporting, and maintainability. It also supports the system requirements for logs, printing, reset tokens, uploads, and archives.

Defense line: “In 3NF, the database becomes clean because each table stores one main idea. Student records, accounts, logs, printing, documents, and archives are separated but connected through keys.”

7. Before-and-After Summary for Defense

View	What Changed	Why It Matters
Before normalization	One logbook-style record can contain student details, category, many requirements, many statuses, many files, and remarks.	Difficult to search, duplicate student/category/requirement data, hard to add new requirements, unclear missing requirements.
After 1NF	Each requirement becomes a separate row.	Repeating columns are removed and values become atomic.
After 2NF	Student data, categories, requirement types, category checklist, and actual student requirement records are separated.	Redundancy is reduced and category-based checklists become easier to manage.
After 3NF	Users, logs, print logs, reset tokens, documents, and archived students are separated into supporting tables.	The database supports accountability, security, report tracking, password recovery, document management, and archiving.
Final ADAMS database	A normalized relational database connected by primary keys, foreign keys, unique constraints, bridge tables, and logical relationships.	Supports accurate retrieval, completeness checking, accountability, and reporting.

8. If Asked: “Is This Based on the Real Database or Just a Sample Table?”

Recommended answer: “The UNF and 1NF tables are sample views used to explain the normalization process. They show how the original manual-style record was transformed. The 2NF and 3NF tables are aligned with the actual ADAMS database structure, including students, student_categories, requirement_types, category_requirements, student_requirements, users, activity_log, print_logs, password_reset_tokens, documents, and archived_students.”

Important distinction:

- System UI screenshots show that the application works.
- Normalization tables show how the database was designed.
- For normalization questions, explain the database/table transformation first. Then connect it to the system features like searching, uploading, monitoring, printing, logging, and archiving.

9. One-Minute Talk Track

Say this if questioned: “Our normalization started from a manual logbook-style admission record. In the unnormalized form, one row could contain student details, category details, and several requirement columns such as Requirement 1, Status 1, File 1, Requirement 2, Status 2, and File 2. The concern was that this structure was repetitive, difficult to maintain, and not flexible when a category requires more documents. In 1NF, we removed the repeating groups by making each requirement a separate row. In 2NF, we separated the data by purpose: students, student categories, requirement types, category requirements, and student requirements. This reduced duplication and allowed the system to apply the correct checklist per category. In 3NF, we separated operational data such as users, activity logs, print logs, password reset tokens, documents, and archived students. This was needed for accountability, security, reporting, and record preservation. The final design is cleaner because each table stores one main idea and is connected through keys and relationships.”

10. Possible Panel Questions and Direct Answers

Question	Best Answer
Q1: What exactly changed from UNF to 1NF?	The repeated columns were removed. Instead of Requirement 1, Requirement 2, Status 1, Status 2, each requirement became a separate row.
Q2: Why was 2NF needed after 1NF?	Because student details, category details, and requirement names were still repeated in the 1NF rows. 2NF separated them into their own tables.
Q3: Why do you need category_requirements?	Because different student categories may require different documents. The bridge table maps requirement types to each category.
Q4: Why was 3NF needed?	Because user actions, print history, password tokens, documents, and archive records are separate operational concerns and should not be stored inside the student table.
Q5: Is the student a login user?	No. In the final scope, only Superadmin and Staff are login users. Students/applicants are admission records maintained by authorized users.
Q6: What is the benefit of the normalized design?	It reduces duplicate data, prevents inconsistency, supports missing requirement reports, improves accountability, and makes the database easier to maintain.

11. Quick Checklist Before Presenting

- Show the UNF table first and point out the repeated requirement columns.
- Show the 1NF table and say: “one requirement per row.”
- Show the 2NF table list and say: “we separated data by purpose.”
- Show the 3NF table list and say: “we separated operational and accountability data.”
- Use the phrase: “The sample tables explain the process; the final tables align with our actual ADAMS database.”

12. Normalization Has a Required Hierarchy

Normalization has a required path: UNF -> 1NF -> 2NF -> 3NF. It is not random. Each stage has a rule that must be satisfied before the next stage can be claimed.

Main defense idea: 3NF already assumes the table is in 1NF and 2NF. If repeating groups still exist, the design is not even 1NF; therefore, it cannot be called 3NF yet.

Stage	Required Before This Stage	What This Stage Checks	Why It Cannot Be Skipped
UNF	None. This is the raw/problem state.	Shows repeated columns, mixed data, and spreadsheet-style storage.	UNF is the starting point used to identify what is wrong.
1NF	Starts from UNF.	Checks if fields are atomic and if repeating groups are removed.	You cannot discuss 2NF/3NF properly if Requirement 1, Requirement 2, Status 1, Status 2 still exist.
2NF	Must already satisfy 1NF.	Checks if main data is separated by purpose and not repeatedly dependent on a combined record.	If student/category/requirement names are still repeated in every row, the design is cleaner but still redundant.
3NF	Must already satisfy 1NF and 2NF.	Checks if indirect or supporting data is separated from the main records.	If logs, print history, reset tokens, and archives are mixed into the main student record, maintenance and accountability become weak.

13. If Asked: Why Cannot We Update Everything Already in 1NF?

Possible professor question: “Why not separate users, logs, print logs, reset tokens, documents, and archives already in 1NF?”

Best answer: In actual system design, we may already plan the final tables early. However, in a normalization explanation, each stage proves a different rule. 1NF only proves that values are atomic and repeating groups are removed. It does not yet prove that all dependencies and supporting records are properly separated.

So, even if we already know the final tables, we explain them stage by stage because each stage answers a different database concern.

Question	Direct Answer for Defense
Can we separate everything in 1NF?	We can plan the final structure early, but academically 1NF only focuses on atomic values and removal of repeating groups. The reasons for separating entities and operational tables belong to 2NF and 3NF.
Why continue to 2NF after 1NF?	Because after 1NF, student details, category names, and requirement names may still repeat across many rows. 2NF separates the core entities.
Why continue to 3NF after 2NF?	Because the database still has supporting concerns such as user accounts, activity logs, print logs, reset tokens, documents, and archives. These should be separate from the main student and requirement records.
What is the safest one-line answer?	1NF fixes the format of the rows; 2NF fixes repeated entity data; 3NF fixes indirect/supporting data and makes the database maintainable.

Defense line: “1NF is correct, but it is not complete. It removes repeating columns, but it does not fully remove redundancy or separate all dependency concerns. That is why we

still need 2NF and 3NF."

14. If Asked: Why Cannot One Requirement per Row Be Done Only in 3NF?

Possible professor question: "If 3NF is the final design, why not place one requirement per row only in 3NF?"

Best answer: Because one requirement per row is the basic requirement of 1NF. 3NF is not a replacement for 1NF. 3NF comes after 1NF and 2NF. If the table still has Requirement 1, Requirement 2, Status 1, Status 2, it fails 1NF and therefore cannot be considered 3NF.

Fix	Correct Stage	Reason
Remove Requirement 1 / Requirement 2 repeating columns	1NF	This fixes repeating groups and makes each field atomic.
Separate students, categories, requirement types, category checklist, and student requirement records	2NF	This fixes repeated core data and separates tables by purpose.
Separate users, logs, print logs, password reset tokens, documents, and archives	3NF	This fixes indirect/supporting data and keeps each table focused on one main idea.

15. "Kahit Pagbalibaligtarin" Review Map

Use this section when the question is asked in a different way. The key is to answer based on the rule of the stage, not only based on the table name.

If the Professor Asks...	What You Should Say
"Why not jump directly from UNF to 3NF?"	In actual design, a developer may design the final schema directly. But for defense, we show the logical normalization path because 3NF assumes 1NF and 2NF are already satisfied.
"Can 1NF already have many tables?"	It may have separated rows or even early table planning, but the 1NF justification is only atomicity and no repeating groups. The full reason for separating entities belongs to 2NF and 3NF.
"Why are logs and print records not discussed in 1NF?"	Because logs and print records are not about atomic values. They are about operational accountability and indirect/supporting data, which is explained under 3NF.
"What if students still repeat in 1NF?"	That is acceptable at 1NF stage because 1NF only removes repeating groups. The repeated student/category data is exactly the reason why we proceed to 2NF.
"What is the hierarchy?"	UNF shows the problem. 1NF fixes repeated columns. 2NF separates core entities. 3NF separates indirect or supporting data.

Memory shortcut: 1NF = row/column format. 2NF = core tables/entities. 3NF = clean dependencies and support tables.

16. Short Oral Answer You Can Memorize

“We cannot put every correction under 1NF because normalization is hierarchical. 1NF only checks atomic values and removes repeating groups. After satisfying 1NF, we move to 2NF to separate repeated core data such as students, categories, and requirement types. After satisfying 2NF, we move to 3NF to separate indirect or supporting data such as users, logs, print history, reset tokens, documents, and archives. So the stages are not random; each stage has its own rule and the next stage assumes the previous stage is already satisfied.”